

Leveransberättelse - Systemutvecklingsprojekt

Delleverans/Slutleverans för KolleganAI

Gruppnummer: 27

Gruppmedlemmar: Dennis Najetovic

Datum: 2026-04-14

Innehållsförteckning

[Infoga innehållsförteckning här.]

T. ex.

Innehåll

1. Översikt

[Kort textuell översikt över projektet, använd beskrivningen från projektplanen, ev. i uppdaterad version.]

Projektet genomförs i samarbete med företaget KolleganAI och syftade initialt till att utveckla en CRM Core Platform som skulle fungera som ett fundament för företagets AI-baserade tjänster. Systemet var tänkt att hantera centrala funktioner såsom autentisering, rollbaserad åtkomstkontroll (RBAC), lead-hantering samt aktivitetsloggning.

Under projektets gång har dock fokus förändrats. Baserat på verksamhetens aktuella behov och prioriteringar beslutades det att istället rikta utvecklingsarbetet mot ett offertsystem. Detta system har som mål att effektivisera processen för att skapa, hantera och följa upp offerter, vilket är en central del i många företags säljprocesser.

Det nya offertsystemet utvecklas fortsatt inom ramen för samma tekniska ekosystem som ursprungligen planerades, med moderna webbutvecklingsteknologier såsom Next.js, TypeScript och en relationsdatabas. Systemet är tänkt att integreras med övriga delar av KolleganAIs plattform och på sikt kunna samverka med AI-agenter för att automatisera delar av offertflödet.

Projektet bedrivs enligt agila principer med Scrum som metodstöd, där arbetet organiseras i sprintar med kontinuerlig uppföljning och iteration. Fokus i denna delleverans ligger på de

analys- och designsteg som genomförts samt de delar av systemet som hittills har implementerats.

2. Ändringar i relation till plan

[Eventuella ändringar (eller preciseringar) i relation till projektplanen. T.ex. ändringar/preciseringar gällande organisation, mål, problem, metoder, kvalitetskriterier, tidplan, resurser, riskanalys.]

I den ursprungliga projektplanen var målet att utveckla en CRM Plattform med funktionalitet för bland annat autentisering, rollbaserad åtkomstkontroll (RBAC), lead-hantering samt aktivitetsloggning. Systemet skulle fungera som en grund för Kollegans AI-agentbaserade lösningar.

Under projektets gång genomfördes dock en omprioritering av projektets inriktning. I dialog med teamet och efter en behovsanalys identifierades att ett offertsystem skulle skapa ett mer direkt affärsvärde i ett tidigare skede, jämfört med att utveckla ett fullskaligt CRM-system. Detta resulterade i ett strategiskt beslut att skifta fokus från CRM-plattformen till utveckling av ett system för hantering av offerter.

Denna förändring innebar en justering av projektets mål och omfattning. Istället för att bygga en bred och generell plattform har arbetet fokuserats på en mer avgränsad men verksamhetskritisk del av säljprocessen.

Även om projektets funktionella inriktning har förändrats, har flera delar av den tekniska och metodmässiga planen kunnat behållas. Den tekniska stacken, med Next.js, TypeScript och PostgreSQL, används fortsatt, och projektet drivs fortfarande enligt agila principer med Scrum som metodstöd. Sprintplanering, dagliga standups och kontinuerlig integration har därmed kunnat fortsätta utan större förändringar.

Förändringen har även påverkat riskbilden i projektet. Genom att minska systemets omfattning har risken för att inte hinna leverera ett fungerande system inom tidsramen reducerats. Samtidigt har nya utmaningar uppstått i form av kravförtydliganden och behovet av att snabbt omdefiniera systemets struktur och funktionalitet.

Sammanfattningsvis har spårbytet inneburit en mer fokuserad och affärsnära utvecklingsprocess, där målet är att leverera ett konkret och användbart system snarare än en bred plattform. Detta bedöms öka sannolikheten att uppnå ett fungerande resultat inom projektets tidsram. Och senare kunna kopplas ihop till CRM systemet.

3. Leverabler

[Här presenteras de leverabler som levereras tillsammans med leveransberättelsen- allt som levereras skall beskrivas. Det skall framgå vad som levereras och vilken roll det har i sammanhanget utan att man som granskare ska behöva tolka eller gissa. Här skall också

tydligt framgå var man hittar materialet – detektivarbete från granskarens sida ska inte behövas.

Om leverablerna är av karaktären analysdokument, t ex produkt backlog, sprint backlog, user stories, användningsfall, klassdiagram etc. *skall* de infogas i denna leveransberättelse (antingen i den löpande texten eller som bilagor till dokumentet om beskrivningen spänner över väldigt många sidor). Är leverablerna mer knutna till systemet, t ex att resultatet är ett nytt inkrement av systemet så beskrivs denna leverabel/inkrement i text med hänvisning till systemet.

Leverablerna skall enkelt uttryckt uppvisa vad som gjorts och har kanske oftast formerna av dokumentation eller mjukvara. För att kunna visa upp leverablerna kan uppfinningsrikedom ibland vara bra – även en leverabel som tycks omöjlig att ”visa upp” behöver på något sätt kunna kommuniceras. Vad gäller dokumentation avses här olika former av analysdokument (ovan). Det kan också handla om systemdokumentation (användarinstruktioner etc.) men även om ”pappersprototyper” eller skärmdumpar som använts under kommunikationen med användare eller beställare. Mjukvaruleverabler kan alltså vara hela eller delar av körbara system, programkod och eller systemprototyper med begränsad funktionalitet.]

Nedan rubriker är förslag på underrubriker för leverablerna.

Frågor som man bör svara på:

1. Har ni kod? Vad funkar just nu?
2. Har ni UI (sidor/komponenter)?
3. Har ni backlog/user stories?
4. Har ni databas (t.ex. Prisma schema)?
5. Har ni något annat (screenshots, API, etc)?

3.1 Produkt-/systembeskrivning

[Här presenteras någon form av övergripande beskrivning av det resultat man nu levererar].

3.2 Applikation/kod och granskningsinstruktioner

[Vanligen behöver delleveransens mottagare instruktioner för att kunna granska leverablerna. Detta avsnitt bör därför innehålla nödvändig information för detta. Det kan handla om information kring hur man gör för att ”köra” ett system (vilka filer), vilken hård- respektive mjukvara som krävs, vilka inställningar som behövs, lösenord och URL-angivelser att använda etc. Liknande frågor kan förstås också behöva besvaras för att mottagarna skall kunna granska bifogad källkod. Syftet är förstås att ni som mottagare inte skall behöva lägga någon kraft alls på att söka information eller fråga om dessa aspekter.]

3.3 Analyser och designspecifikationer

[T ex produkt backlog, sprint backlog, user stories, användningsfall, klassdiagram etc.]

3.4 Mjukvaruarkitektur- beskrivning, motiv och reflektion

[När slutprodukten i fråga innehåller mjukvara behandlas det utvecklade systemets arkitektur här. Det vill säga en beskrivning av exempelvis lager/och eller skiktindelningar och

klasstrukturer. Beskrivningen av arkitekturen relateras där så är möjligt till etablerade designmönster. De gjorda arkitekturvalen ska även förklaras och motiveras. Här är det viktigt att visa upp sin medvetenhet om de gjorda valen, deras konsekvens och lämplighet för slutprodukten. Alternativt, om något val visat sig mindre lämpligt, skall en reflektion över detta presenteras.]

3.5 Kodstruktur, förvaltningsbarhet & återanvändning, driftsäkerhet

[Här beskriver ni hur ni har arbetat med kodstruktur under arbetet (syntax, namnsättning, kodkommentarer etc.) samt hur ni har arbetat för att säkerställa förvaltningsbarhet och om ni återanvänt befintlig kod och/eller anpassat er till ett befintligt system. Även hur ni har arbetat med felhantering för att säkerställa driften av systemet beskrivs.]

3.6 Kvalitetskriterier

[Här beskrivs hur de planerade kvalitetskriterierna har hanterats, hur har de identifierade kvalitetskriterierna, från projektplanen, påverkat arbetet? Eller har andra/nya kvalitetskrav påverkat arbetet? En kvalitet som ofta påverkar designprocessen är kravet på användbarhet – hur har ni jobbat med att säkra användbarhet?]

4. Reflektion

4.1 Val av utvecklingsverktyg

[Här beskrivs de utvecklingsverktyg som utnyttjats under processen samt reflektioner om hur väl valda/eller givna verktyg fungerat i arbetet. Motiv till olika val som gjorts skall presenteras. Det som ska kunna bedömas är er förståelse och reflektion över konsekvenserna av valda utvecklingsverktyg. I de fall en uppdragsgivare bestämt utvecklingsverktyg åt er reflekterar ni över hur detta har påverkat er process och arbete och om det skulle ha kunnat finnas mer lämpliga val.]

Under projektets gång har ett antal utvecklingsverktyg och teknologier använts för att möjliggöra en effektiv och strukturerad utvecklingsprocess. Valet av verktyg baserades både på projektets tekniska krav och på uppdragsgivarens befintliga tekniska miljö.

I den ursprungliga planen, där fokus låg på att utveckla ett CRM-system, valdes teknologier som Next.js och TypeScript för att möjliggöra utveckling av en skalbar fullstack-applikation. Detta var särskilt viktigt då CRM-systemet innehöll flera komplexa komponenter, såsom rollbaserad åtkomstkontroll (RBAC), lead-hantering och aktivitetsloggning, vilket ställde höga krav på struktur och kodkvalitet.

Efter spårbytet till ett offertsystem har dessa verktyg fortsatt att användas, men i ett något annorlunda sammanhang. Offertsystemet har ett mer avgränsat fokus jämfört med CRM-systemet, vilket har gjort det möjligt att arbeta mer iterativt med frontend-komponenter och användarflöden kopplade till offertskapande. TypeScript har fortsatt varit värdefullt för att säkerställa korrekt datahantering, särskilt vid hantering av offertdata och kundinformation.

För databashantering har PostgreSQL i kombination med Prisma ORM använts i båda fallen. I CRM-systemet var datamodellen mer omfattande, med flera relationer mellan exempelvis användare, leads och aktiviteter. I offertsystemet har datamodellen istället fokuserats kring offerter, kunder och relaterad information, vilket har gjort strukturen mer lättöverskådlig men samtidigt ställt krav på flexibilitet för framtida utbyggnad.

Versionshantering via GitHub samt användning av GitHub Projects för backloghantering har varit centralt genom hela projektet, oavsett systemets inriktning. Dessa verktyg har bidragit till en tydlig struktur och underlättat samarbete inom teamet. Kommunikation via Slack och Discord har även varit en viktig del i att hantera både tekniska frågor och förändringar i projektets riktning.

Sammanfattningsvis har de valda utvecklingsverktygen visat sig vara väl lämpade för både det ursprungliga CRM-systemet och det nuvarande offertsystemet. Skillnaden ligger främst i hur verktygen har använts, där fokus har gått från att hantera komplex systemarkitektur till att möjliggöra snabb och flexibel utveckling av en mer avgränsad applikation.

4.2 Metodreflektion

[Här beskrivs det systemutvecklingsmetodstöd som utnyttjats under processen samt reflektioner om hur väl vald metod fungerat i arbetet. Här ingår även att reflektera över användning av eventuell mjukvara för process-/metodstöd. Det är viktigt att genomförandet av utvecklingsarbetet beskrivs och värderas. Fokus riktas mot hur olika metodkomponenter använts samt utfallet av att använda dem. Med metodreflektion avses förstås också att man diskuterar det tänkbara utfallet av att ha gjort på ett alternativt sätt.]

Projektet har genomförts med Scrum som metodstöd, vilket har inneburit ett iterativt och inkrementellt arbetssätt. Arbetet har organiserats i sprintar med återkommande ceremonier såsom sprintplanering, dagliga standups, sprint reviews och retrospektiv.

I den ursprungliga planen, där fokus låg på utvecklingen av ett CRM-system, var Scrum särskilt lämpligt då projektet hade en hög grad av komplexitet och många beroenden mellan olika funktionella delar. Genom att dela upp arbetet i sprintar kunde teamet successivt bygga upp systemets olika komponenter, såsom autentisering, RBAC och lead-hantering, samtidigt som kontinuerlig feedback möjliggjordes.

Efter spårbytet till offertsystemet har Scrum fortsatt att användas, men med ett något förändrat fokus. Offertsystemet är mer avgränsat och användarcentrerat, vilket har gjort det möjligt att arbeta med kortare feedback-loopar och snabbare iterationer kring specifika funktioner, exempelvis skapande och hantering av offerter. Detta har bidragit till en mer direkt koppling mellan utvecklingsarbete och verksamhetsnytta.

Spårbytet i sig illustrerar en av de centrala styrkorna med agila metoder, nämligen förmågan att hantera förändrade krav. Genom att arbeta enligt Scrum kunde projektgruppen relativt smidigt omprioritera och anpassa backloggen utan att behöva omstrukturera hela projektet.

Samtidigt har förändringen också medfört vissa utmaningar. Planerade sprintmål kopplade till CRM-systemet har behövt omdefinieras, vilket i vissa fall kan ha påverkat kontinuiteten i arbetet. Detta har ställt krav på tydlig kommunikation inom teamet samt en nära dialog med uppdragsgivaren för att säkerställa att nya prioriteringar är korrekt förstådda.

Sammanfattningsvis har Scrum fungerat väl som metodstöd både före och efter spårbytet. Metoden har möjliggjort både struktur i ett komplext CRM-projekt och flexibilitet i övergången till ett mer fokuserat offertsystem. Detta visar på vikten av att välja en metodik som stödjer förändring i dynamiska projektmiljöer.

5. Källförteckning

[Infoga de källor som använts i leveransberättelsen.]

Görling, S. (2009). *Att arbeta med IT-projekt*. Studentlitteratur.

Scrum Guides (2020). *The Scrum Guide*. Hämtad från <https://scrumguides.org/>

International Organization for Standardization (2011). *ISO/IEC 25010: Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE)*.

KolleganAI. (2026). *Intern projektplan och kravspecifikation (SRS v3.0)*.

Bilagor

[Här infogas eventuella bilagor]